Computer Architecture
Assignment 4 Guide

Upgrading the ToyRISC Simulator
from Single-Cycle to a
5-Stage Pipelined Processor

A Comprehensive Step-by-Step Guide

Computer Architecture Laboratory

Contents

1 Assignment Overview **2**
1.1 What You Are Asked To Do . . . 2
1.2 Submission Requirements . . . 2

2 Background: Single-Cycle vs. Pipelined Execution **2**
2.1 The Single-Cycle Model (Assignment 3) . . . 2
2.2 The Pipelined Model (Assignment 4) . . . 3

3 Architecture of the Existing Codebase **3**
3.1 Directory Structure . . . 3
3.2 Pipeline Stage Flow Diagram . . . 4
3.3 Latch Types Summary . . . 4
3.4 How Each Stage Works (Current Implementation) . . . 5
3.4.1 Instruction Fetch (IF) — `InstructionFetch.java` . . . 5
3.4.2 Operand Fetch (OF) — `OperandFetch.java` . . . 5
3.4.3 Execute (EX) — `Execute.java` . . . 5
3.4.4 Memory Access (MA) — `MemoryAccess.java` . . . 5
3.4.5 Register Write (RW) — `RegisterWrite.java` . . . 5

4 Step-by-Step Implementation Guide **6**
4.1 Step 1: Modify the Simulation Loop in `Simulator.java` . . . 6
4.2 Step 2: Fix the Enable/Disable Latch Mechanism . . . 6
4.2.1 Modifying `RegisterWrite.java` . . . 7
4.2.2 Modifying Each Stage's Enable Logic . . . 7
4.3 Step 3: Ensure IF Stage Is Always Active . . . 8

5 Implementing Data Interlocks (Hazard Detection and Stalling) **9**
5.1 What is a Data Hazard? . . . 9
5.2 Where to Detect Hazards . . . 9
5.3 How to Track Destination Registers of In-Flight Instructions . . . 9
5.3.1 Approach A: Check the Latches Directly . . . 9
5.3.2 Approach B: Maintain a "Busy" Bit Array (Scoreboard) . . . 9
5.4 Implementing the Stall . . . 10
5.5 Adding Busy Bits to `RegisterFile.java` . . . 11
5.6 Clearing Busy Bits in RW Stage . . . 12

6 Implementing Control Interlocks (Branch Handling) **12**
6.1 What is a Control Hazard? . . . 12
6.2 Pipeline Timing of a Branch . . . 12
6.3 Implementing the Flush . . . 12
6.4 Counting Wrong-Path Instructions . . . 13

7 Modifications to `Statistics.java` **13**

8 Handling the `end` Instruction in a Pipeline **15**

9 Complete Modified File Summary **16**

10 Handling the PC Correctly in a Pipeline **16**

11 Testing and Verification **17**
11.1 Test Cases 17
11.2 How to Build and Run 17
11.3 Using the Test Script 18
11.4 Verification Checklist 18

12 Writing the Report **18**
12.1 Required Table 18
12.2 Observations to Comment On 18

13 Common Pitfalls and Debugging Tips **19**

14 Summary of Changes at a Glance **20**

A ToyRISC ISA Quick Reference **21**
A.1 Instruction Formats 21
A.2 Complete Opcode Table 21
A.3 Registers 22

B Sample Benchmark: Fibonacci Walkthrough **22**

1 Assignment Overview

1.1 What You Are Asked To Do

Assignment 4 requires you to **upgrade the existing single-cycle ToyRISC simulator** (from Assignment 3) into a **pipelined core model**. Specifically:

1. **Convert the simulation loop** from a sequential, one-instruction-at-a-time model to a pipelined model where multiple instructions are in-flight simultaneously.
2. **Implement data interlocks** (stalling the pipeline when a data hazard is detected).
3. **Implement control interlocks** (handling branch/jump instructions that may cause wrong-path instructions to enter the pipeline).

1.2 Submission Requirements

- A zip of the source files that pass all the test cases from Assignment 3.
- A **report** containing:
  - A table with the **number of cycles** taken by each benchmark program.
  - The **number of times the OF stage needed to stall** because of a data hazard.
  - The **number of times an instruction on a wrong branch path entered the pipeline**.
- Comments on observations and correlation with benchmark nature.

2 Background: Single-Cycle vs. Pipelined Execution

2.1 The Single-Cycle Model (Assignment 3)

In the current `Simulator.java`, the simulation loop processes **one instruction fully through all 5 stages** before starting the next:

```java
while(simulationComplete == false) {
    processor.getIFUnit().performIF();
    Clock.incrementClock();
    processor.getOFUnit().performOF();
    Clock.incrementClock();
    processor.getEXUnit().performEX();
    Clock.incrementClock();
    processor.getMAUnit().performMA();
    Clock.incrementClock();
    processor.getRWUnit().performRW();
    Clock.incrementClock();
}
```

Listing 1: Current single-cycle simulation loop

Here, each instruction takes 5 clock cycles, and **no overlap** exists between instructions.

2.2 The Pipelined Model (Assignment 4)

In a pipelined processor, **all five stages operate simultaneously** on different instructions. The new loop must look like:

```
while(not end of simulation) {
    performRW();    // Stage 5
    performMA();    // Stage 4
    performEX();    // Stage 3
    performOF();    // Stage 2
    performIF();    // Stage 1
    increment clock by 1;
}
```

Listing 2: Target pipelined simulation loop

Key Point

The stages are called in **reverse order** (RW → MA → EX → OF → IF) in each clock cycle. This is crucial because each stage reads from its input latch and writes to its output latch. By processing later stages first, you avoid overwriting latch data before it is consumed.

Warning / Common Pitfall

If you call stages in forward order (IF → OF → ...), the IF stage will write to the IF_OF latch, and then the OF stage will immediately read this *new* data in the *same* cycle — violating the rule that each stage operates on the *previous* cycle's output.

3 Architecture of the Existing Codebase

3.1 Directory Structure

```
supporting_files/
|-- build.xml
|-- src/
|    |-- configuration/
|    |    |-- Configuration.java
|    |    |-- config.xml
|    |-- generic/
|    |    |-- Instruction.java
|    |    |-- Misc.java
|    |    |-- Operand.java
|    |    |-- Simulator.java
|    |    |-- Statistics.java
|    |-- main/
|    |    |-- Main.java
|    |-- processor/
|         |-- Clock.java
|         |-- Processor.java
|         |-- memorysystem/
|         |    |-- MainMemory.java
```

```
|        |-- pipeline/
|            |-- EX_IF_LatchType.java
|            |-- EX_MA_LatchType.java
|            |-- Execute.java
|            |-- IF_EnableLatchType.java
|            |-- IF_OF_LatchType.java
|            |-- InstructionFetch.java
|            |-- MA_RW_LatchType.java
|            |-- MemoryAccess.java
|            |-- OF_EX_LatchType.java
|            |-- OperandFetch.java
|            |-- RegisterFile.java
|            |-- RegisterWrite.java
|-- test_cases/
    |-- descending.asm, fibonacci.asm, ...
```

Listing 3: Project directory layout

3.2 Pipeline Stage Flow Diagram

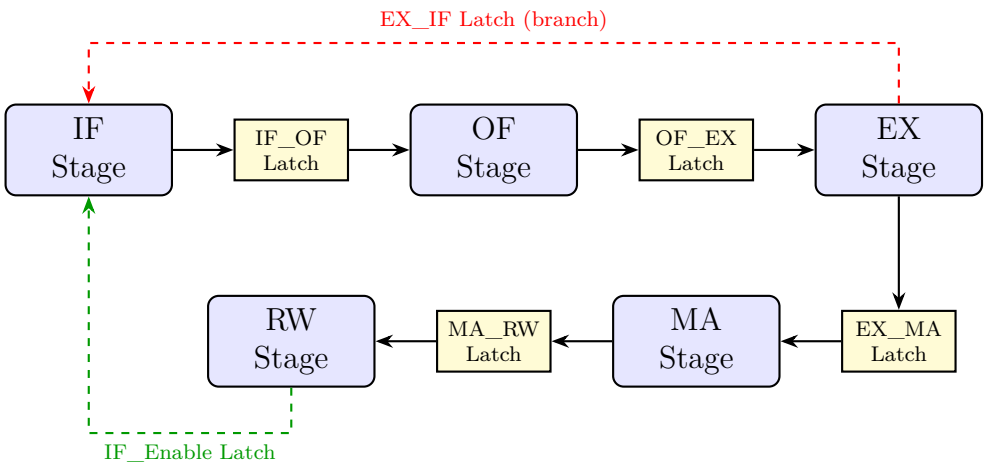

Figure 1: 5-stage pipeline with inter-stage latches and feedback paths

3.3 Latch Types Summary

Latch	Contents	Connects
`IF_EnableLatchType`	`IF_enable` (boolean)	RW → IF
`IF_OF_LatchType`	`instruction` (int), `OF_enable`	IF → OF
`OF_EX_LatchType`	`instruction` (Instruction), `EX_enable`	OF → EX
`EX_MA_LatchType`	`instruction`, `aluResult`, `MA_enable`	EX → MA
`EX_IF_LatchType`	`isBranchEnabled`, `pc`	EX → IF
`MA_RW_LatchType`	`instruction`, `aluResult`, `Idresult`, `RW_enable`	MA → RW

Table 1: Summary of all inter-stage latches

3.4 How Each Stage Works (Current Implementation)

3.4.1 Instruction Fetch (IF) — `InstructionFetch.java`

- Checks `IF_EnableLatch.isIF_enable()`.
- If a branch was taken (from `EX_IF_Latch`), updates PC accordingly.
- Reads the instruction word from `MainMemory[PC]`.
- Writes the raw instruction integer to `IF_OF_Latch`.
- Increments PC by 1.

3.4.2 Operand Fetch (OF) — `OperandFetch.java`

- Decodes the 32-bit binary instruction (opcode, operands).
- Creates an `Instruction` object with `OperationType`, source and destination `Operand`s.
- Handles R3-Type, R2I-Type, RI-Type formats and sign extension for immediates.
- Writes the decoded instruction to `OF_EX_Latch`.

3.4.3 Execute (EX) — `Execute.java`

- Reads register values from the `RegisterFile`.
- Performs the ALU operation based on instruction type.
- For branches/jumps: computes target PC, sets `EX_IF_Latch.setBranchEnabled(true)`.
- Passes ALU result to `EX_MA_Latch`.

3.4.4 Memory Access (MA) — `MemoryAccess.java`

- For `load`: reads from `MainMemory[aluResult]` and stores in latch's `Idresult`.
- For `store`: writes register value to `MainMemory[aluResult]`.
- Passes instruction and results to `MA_RW_Latch`.

3.4.5 Register Write (RW) — `RegisterWrite.java`

- For arithmetic instructions: writes ALU result to destination register.
- For `load`: writes loaded value to destination register.
- For `end`: calls `Simulator.setSimulationComplete(true)`.
- Re-enables IF stage by setting `IF_EnableLatch.setIF_enable(true)`.

4 Step-by-Step Implementation Guide

4.1 Step 1: Modify the Simulation Loop in `Simulator.java`

This is the **most critical change**. Replace the single-cycle sequential loop with a pipelined loop.

```java
public static void simulate() {
    while (simulationComplete == false) {
        processor.getRWUnit().performRW();
        processor.getMAUnit().performMA();
        processor.getEXUnit().performEX();
        processor.getOFUnit().performOF();
        processor.getIFUnit().performIF();
        Clock.incrementClock();
    }

    // Set statistics after simulation
    Statistics.setNumberOfInstructions(instructionCount);
    Statistics.setNumberOfCycles(cycleCount);
    Statistics.setCpi(instructionCount, cycleCount);
    Statistics.setIpc(instructionCount, cycleCount);
}
```

Listing 4: New pipelined simulation loop in `Simulator.java`

Key Point

Key changes:

1. Stages are in **reverse order**: RW, MA, EX, OF, IF.
2. Clock is incremented only **once** per iteration (not 5 times).
3. The enable/disable mechanism in latches now controls which stages are "active" in each cycle.

4.2 Step 2: Fix the Enable/Disable Latch Mechanism

In the single-cycle model, the RW stage disables itself and enables IF, creating a strict chain: IF $\rightarrow$ OF $\rightarrow$ EX $\rightarrow$ MA $\rightarrow$ RW $\rightarrow$ IF $\rightarrow \ldots$

In the pipelined model, all stages should be active simultaneously. This means:

- `IF_EnableLatch` should always be `true` (IF fetches every cycle unless stalled).
- Each latch enable bit indicates whether the *downstream* stage has valid data to process.

Warning / Common Pitfall

Do NOT remove the enable bits. They are essential for handling:

- Pipeline startup (first few cycles where later stages have no valid data).
- Stall cycles (when a stage is paused).
- Pipeline flush (when wrong-path instructions must be discarded).

4.2.1 Modifying `RegisterWrite.java`

In the single-cycle model, RW re-enables IF:

```
IF_EnableLatch.setIF_enable(true); // Remove or change this
```

In the pipelined model, IF should **always remain enabled** unless the pipeline is stalled. So in the constructor or the simulation setup, `IF_enable` should default to `true`, and the RW stage should **not** toggle it.

```
public void performRW() {
    if (MA_RW_Latch.isRW_enable()) {
        Instruction instruction = MA_RW_Latch.getInstruction();
        String op = instruction.getOperationType().toString();

        if (op.equals("load")) {
            containingProcessor.getRegisterFile().setValue(
                instruction.getDestinationOperand().getValue(),
                MA_RW_Latch.getIdresult());
        } else if (op.equals("end")) {
            Simulator.setSimulationComplete(true);
        } else if (!(op.equals("store") || op.equals("jmp")
                || op.equals("beq") || op.equals("blt")
                || op.equals("bne") || op.equals("bgt"))) {
            containingProcessor.getRegisterFile().setValue(
                instruction.getDestinationOperand().getValue(),
                MA_RW_Latch.getALu());
        }
        MA_RW_Latch.setRW_enable(false);
        // Do NOT set IF_EnableLatch here in pipelined mode!
    }
}
```

Listing 5: Modified RW stage (key change)

4.2.2 Modifying Each Stage's Enable Logic

Each stage should:

1. Check its input latch's enable bit.
2. If enabled, process and write to the output latch (setting the next stage's enable to `true`).
3. If not enabled (e.g. bubble/NOP), set the output latch enable to `false` (propagate the bubble).

```java
public void performXX() {
    if (inputLatch.isXX_enable()) {
        // Process the instruction
        // Write results to output latch
        outputLatch.setNextEnable(true);
    } else {
        // Propagate bubble
        outputLatch.setNextEnable(false);
    }
}
```

Listing 6: General pattern for each pipeline stage

4.3 Step 3: Ensure IF Stage Is Always Active

In the pipelined model, the IF stage should run every cycle (unless stalled). Modify the initialization:

```java
// In Processor constructor or Simulator.setupSimulation:
IF_EnableLatch.setIF_enable(true);
// This is already the default in the constructor.

// In InstructionFetch.performIF():
// Do NOT set IF_EnableLatch.setIF_enable(false) at the end!
// The IF stage should remain enabled for the next cycle.
```

Listing 7: IF stage always enabled

```java
public void performIF() {
    if (IF_EnableLatch.isIF_enable()) {
        if (EX_IF_Latch.isBranchEnabled()) {
            int newPC = EX_IF_Latch.getPC();
            containingProcessor.getRegisterFile()
                .setProgramCounter(newPC);
            EX_IF_Latch.setBranchEnabled(false);
        }
        int currentPC = containingProcessor.getRegisterFile()
            .getProgramCounter();
        int newInstruction = containingProcessor.getMainMemory()
            .getWord(currentPC);
        IF_OF_Latch.setInstruction(newInstruction);
        containingProcessor.getRegisterFile()
            .setProgramCounter(currentPC + 1);
        IF_OF_Latch.setOF_enable(true);
        // DO NOT disable IF_enable!
    }
}
```

Listing 8: Modified IF stage for pipelining

5 Implementing Data Interlocks (Hazard Detection and Stalling)

5.1 What is a Data Hazard?

A **data hazard** occurs when an instruction in the OF stage needs to read a register that a preceding instruction (currently in EX, MA, or RW) has not yet written back to. This is a **Read-After-Write (RAW)** hazard.

Example:

```
addi %x0, 5, %x3      ; writes to x3
add  %x3, %x4, %x5    ; reads x3 -- DATA HAZARD!
```

When the `add` instruction is in the OF stage trying to read `x3`, the `addi` instruction may still be in EX, MA, or RW and has *not yet* written the new value of `x3` to the register file.

5.2 Where to Detect Hazards

Detect hazards in the **OF stage** (`OperandFetch.java`). The OF stage decodes the instruction and knows which registers it needs. It must check if any instruction in EX, MA, or RW stages will write to those registers.

5.3 How to Track Destination Registers of In-Flight Instructions

You need to know what registers are being written by instructions currently in the pipeline. Two common approaches:

5.3.1 Approach A: Check the Latches Directly

Inspect the `OF_EX_Latch`, `EX_MA_Latch`, and `MA_RW_Latch` to read the instructions in later stages and check their destination registers.

5.3.2 Approach B: Maintain a "Busy" Bit Array (Scoreboard)

Maintain an array `boolean[] registerBusy = new boolean[32]` in the `RegisterFile`:

- When OF dispatches an instruction that writes to register `rd`, set `registerBusy[rd] = true`.
- When RW writes back to register `rd`, set `registerBusy[rd] = false`.
- In OF, before reading source registers, check if `registerBusy[rs1]` or `registerBusy[rs2]` is true. If so, **stall**.

Key Point

Approach B (scoreboard) is simpler and is **recommended** for this assignment.

5.4 Implementing the Stall

When a data hazard is detected in OF:

1. **OF does NOT write to OF_EX latch** — it keeps its input unchanged.
2. **OF_EX_Latch enable is set to false** — inject a "bubble" (NOP) into EX.
3. **IF is stalled** — the IF stage must not fetch a new instruction. Set `IF_EnableLatch.setIF_enabl`
4. **Decrement PC** back by 1, or simply do not let IF advance. The simplest approach: just disable IF by not enabling it.

```java
public void performOF() {
    if (IF_OF_Latch.isOF_enable()) {
        // Decode the instruction as before...
        Instruction inst = decode(IF_OF_Latch.getInstruction());

        // Identify source registers
        int src1Reg = -1, src2Reg = -1;
        if (inst.getSourceOperand1() != null
            && inst.getSourceOperand1().getOperandType()
               == OperandType.Register) {
            src1Reg = inst.getSourceOperand1().getValue();
        }
        if (inst.getSourceOperand2() != null
            && inst.getSourceOperand2().getOperandType()
               == OperandType.Register) {
            src2Reg = inst.getSourceOperand2().getValue();
        }

        // Check for hazards (busy registers)
        RegisterFile rf = containingProcessor.getRegisterFile();
        boolean hazard = false;
        if (src1Reg >= 0 && rf.isRegisterBusy(src1Reg))
            hazard = true;
        if (src2Reg >= 0 && rf.isRegisterBusy(src2Reg))
            hazard = true;

        if (hazard) {
            // STALL: inject bubble into EX
            OF_EX_Latch.setEX_enable(false);
            // Keep IF stalled (do not fetch next instruction)
            IF_EnableLatch.setIF_enable(false);
            // Increment data hazard stall counter
            Statistics.incrementDataHazardStalls();
        } else {
            // No hazard: proceed normally
            OF_EX_Latch.setInstruction(inst);
            OF_EX_Latch.setEX_enable(true);
            IF_OF_Latch.setOF_enable(false);

            // Mark destination register as busy
            int destReg = getDestRegister(inst);
            if (destReg > 0) { // don't mark x0
                rf.setRegisterBusy(destReg, true);
            }
```

```
        }
    } else {
        // No valid instruction: propagate bubble
        OF_EX_Latch.setEX_enable(false);
    }
}
```

Listing 9: Hazard detection and stalling in OF stage

Warning / Common Pitfall

Special cases for hazard detection:

- `store` instructions: the "destination" operand in the Instruction object is actually the register whose contents provide the memory address. The actual register to check for reads is `sourceOperand1` (rs1). Be careful about which registers count as "sources."
- `jmp` instructions: may read from a register in the destination operand field. Handle accordingly.
- Branches (`beq`, `bne`, `blt`, `bgt`): both `sourceOperand1` and `sourceOperand2` are registers to compare.
- `x0` is hardwired to 0 and should never be marked as busy.
- `x31` may be written implicitly (by `div`, `mul`, shift instructions). Consider whether to mark it busy.

5.5 Adding Busy Bits to `RegisterFile.java`

```
public class RegisterFile {
    int[] registerFile;
    int programCounter;
    boolean[] busy; // NEW

    public RegisterFile() {
        registerFile = new int[32];
        busy = new boolean[32]; // all false initially
    }

    public boolean isRegisterBusy(int regNum) {
        return busy[regNum];
    }

    public void setRegisterBusy(int regNum, boolean val) {
        if (regNum != 0) // x0 is always 0, never busy
            busy[regNum] = val;
    }
    // ... existing methods unchanged ...
}
```

Listing 10: Additions to `RegisterFile.java`

5.6 Clearing Busy Bits in RW Stage

In `RegisterWrite.java`, after writing the result to the register, clear the busy bit:

```
// After writing to register rd:
containingProcessor.getRegisterFile()
    .setRegisterBusy(rd, false);
```

Listing 11: Clearing busy bits in RW stage

6 Implementing Control Interlocks (Branch Handling)

6.1 What is a Control Hazard?

A **control hazard** occurs when a branch or jump instruction is in the pipeline, and the processor has already fetched subsequent instructions that may be on the **wrong execution path**.

In this simulator, branches are resolved in the **EX stage**. By the time EX determines that a branch is taken, the IF and OF stages have already fetched and decoded 1–2 instructions from the **fall-through path** (the path assuming the branch is not taken).

6.2 Pipeline Timing of a Branch

Cycle	**1**	**2**	**3**	**4**	**5**	**6**
Branch instr.	IF	OF	EX	MA	RW	
Instr. at PC+1		IF	OF	*flush*		
Instr. at PC+2			IF	*flush*		
Target instr.				IF	OF	EX

Table 2: Branch instruction timing in a 5-stage pipeline

When the branch is resolved as **taken** in cycle 3 (EX stage), the two instructions already fetched (at PC+1 and PC+2) must be **flushed** (invalidated).

6.3 Implementing the Flush

When EX detects a taken branch:

1. Set `EX_IF_Latch.setBranchEnabled(true)` and `EX_IF_Latch.setPC(targetPC)`.
2. **Invalidate the IF_OF latch** (the instruction in OF is on the wrong path): set `IF_OF_Latch.setOF_enable(false)`.
3. **Invalidate the OF_EX latch** (the instruction about to enter EX is also wrong): set `OF_EX_Latch.setEX_enable(false)`.

```java
// When a branch is taken (e.g., beq condition is true):
alu = pcnow + immed;
EX_IF_Latch.setBranchEnabled(true);
EX_IF_Latch.setPC(alu);

// Flush pipeline: invalidate instructions in IF and OF
IF_OF_Latch.setOF_enable(false);
// The OF_EX_Latch will be set to false by default
// since OF won't produce valid output

// Increment wrong-path counter
Statistics.incrementWrongBranchCount();
```

Listing 12: Branch handling in EX stage

Key Point

For **unconditional jumps** (`jmp`), the branch is **always taken**, so you always flush. For conditional branches, you only flush when the condition evaluates to true.

Tip

The number of wrong-path instructions that entered the pipeline corresponds to how many instructions were fetched and started processing before the branch was resolved. In a simple 5-stage pipeline with branch resolution at EX, this is typically **2 instructions** per taken branch (the ones in IF and OF stages).

6.4 Counting Wrong-Path Instructions

Each time a branch is taken and the pipeline is flushed, count how many instructions on the wrong path entered the pipeline. This is the number of valid instructions in stages between IF and EX at the time of the branch resolution:

```java
// In Execute.java, when branch is taken:
int wrongPathCount = 0;
if (IF_OF_Latch.isOF_enable()) wrongPathCount++;
// The instruction that was in IF during this cycle
// (it will have been fetched before we noted the branch)
wrongPathCount++; // IF always fetches if enabled

Statistics.addWrongBranchInstructions(wrongPathCount);
```

Listing 13: Counting wrong branch entries

7 Modifications to `Statistics.java`

Add new counters for the required statistics:

```java
public class Statistics {
    static int numberOfInstructions;
    static int numberOfCycles;
```

```java
    static float ipc;
    static float cpi;
    static int dataHazardStalls;     // NEW
    static int wrongBranchCount;     // NEW

    // Increment methods
    public static void incrementDataHazardStalls() {
        dataHazardStalls++;
    }

    public static void incrementWrongBranchCount() {
        wrongBranchCount++;
    }

    // Getters
    public static int getDataHazardStalls() {
        return dataHazardStalls;
    }

    public static int getWrongBranchCount() {
        return wrongBranchCount;
    }

    public static void printStatistics(String statFile) {
        try {
            PrintWriter writer = new PrintWriter(statFile);
            writer.println("Number of instructions = "
                + numberOfInstructions);
            writer.println("Number of cycles = "
                + numberOfCycles);
            writer.println("CPI = " + cpi);
            writer.println("IPC = " + ipc);
            writer.println("Data hazard stalls = "
                + dataHazardStalls);
            writer.println("Wrong branch instructions = "
                + wrongBranchCount);
            writer.close();
        } catch (Exception e) {
            Misc.printErrorAndExit(e.getMessage());
        }
    }
    // ... existing setters remain ...
}
```

Listing 14: Updated `Statistics.java`

8 Handling the `end` Instruction in a Pipeline

Warning / Common Pitfall

In the pipelined model, when an `end` instruction is fetched, the simulator must **not stop immediately**. The `end` instruction must flow through all 5 stages (IF → OF → EX → MA → RW) before `setSimulationComplete(true)` is called. Meanwhile, the IF stage may continue fetching instructions after `end`, but those are irrelevant as long as the simulation terminates correctly when `end` reaches RW.

One approach: when the OF stage decodes an `end` instruction, disable IF to prevent further fetches:

```
case end:
    // In OF stage: stop fetching
    IF_EnableLatch.setIF_enable(false);
    break;
```

The `end` instruction then propagates through EX, MA, and RW. When RW processes it, it calls `Simulator.setSimulationComplete(true)`.

9 Complete Modified File Summary

File	Changes Required
`Simulator.java`	Rewrite `simulate()` to call stages in reverse order with a single `Clock.incrementClock()` per iteration.
`InstructionFetch.java`	Do not disable `IF_enable` at end of `performIF()`. Handle stall signal (when OF requests a stall, IF must not advance PC).
`OperandFetch.java`	Add hazard detection logic. Check busy registers. If hazard: stall (inject bubble, disable IF). Add counter increment for data hazard stalls.
`Execute.java`	On taken branch: flush IF_OF and OF_EX latches. Increment wrong-path counter. Fix pcnow calculation for pipelined operation.
`MemoryAccess.java`	Propagate bubbles (if MA_enable is false, set RW_enable false).
`RegisterWrite.java`	Remove `IF_EnableLatch.setIF_enable(true)`. Clear busy bits after writeback.
`RegisterFile.java`	Add `boolean[] busy` array with `isRegisterBusy()` and `setRegisterBusy()` methods.
`Statistics.java`	Add `dataHazardStalls` and `wrongBranchCount` counters with increment methods and print support.
`IF_OF_LatchType.java`	Possibly add a field to store the PC of the fetched instruction (useful for branch target calculation).
`OF_EX_LatchType.java`	Possibly add a field to store the PC (for EX stage branch target computation).

Table 3: Summary of all files to modify

10 Handling the PC Correctly in a Pipeline

In the single-cycle model, the EX stage computes `pcnow` as:

```
int pcnow = containingProcessor.getRegisterFile()
    .programCounter - 1;
```

This works because in the single-cycle model, PC was just incremented by IF and no other instruction has touched it.

Warning / Common Pitfall

In the pipelined model, **PC keeps advancing every cycle** as IF fetches new instructions. By the time an instruction reaches EX, the PC has already moved ahead by 2 or more. The method above will give the **wrong** value.

Solution: Store the PC of each instruction **in the latches** as it flows through the pipeline.

1. In `IF_OF_LatchType`, add a field `int instructionPC` and set it during IF.

2. Carry this PC through each latch: OF_EX, EX_MA, MA_RW.
3. In the Instruction object itself, use the existing `programCounter` field — set it during OF.

```
// In OperandFetch.performOF():
inst.setProgramCounter(IF_OF_Latch.getInstructionPC());
```

Listing 15: Storing PC in the instruction during OF

```
// In Execute.performEX():
int pcnow = inst.getProgramCounter();
// Branch target: alu = pcnow + immed;
```

Listing 16: Using stored PC in EX for branch target

11 Testing and Verification

11.1 Test Cases

The provided test cases (from Assignment 3) must still produce correct results:

Benchmark	Description
`fibonacci.asm`	Computes first 10 Fibonacci numbers, stores them on the stack. Loop-heavy with moderate data dependencies.
`descending.asm`	Bubble sort of 8 elements in descending order. Nested loops with many branches and data dependencies on loaded values.
`prime.asm`	Checks if 11 is prime by trial division. Uses `div` and checks `x31` (remainder).
`evenorodd.asm`	Checks if 11 is even or odd. Very short, uses `divi` and `beq`.
`palindrome.asm`	Checks if 4567654 is a palindrome. Loop with `divi`/`muli` and `x31`.

Table 4: Benchmark programs and their characteristics

11.2 How to Build and Run

```
# Build
cd supporting_files
ant build

# Create JAR
ant make-jar

# Run on a test case
java -jar jars/simulator.jar \
    src/configuration/config.xml \
```

```
    output_stats.txt \
    test_cases/fibonacci.out

# Check hash value against expected
cat test_cases/fibonacci.expected
```

Listing 17: Building and running the simulator

11.3 Using the Test Script

```
python test_zip.py <path-to-your-zip>
```

11.4 Verification Checklist

- ☐ All 5 test cases produce the **correct hash value** (same as Assignment 3).
- ☐ The **final register and memory states** are identical to the single-cycle model.
- ☐ The cycle count is $\geq$ the instruction count (pipeline overhead from hazards).
- ☐ Data hazard stall count is reasonable:
  - `evenorodd`: few stalls (very short program).
  - `descending`: many stalls (loads followed by comparisons).
- ☐ Wrong-branch-path count correlates with number of taken branches.

12 Writing the Report

12.1 Required Table

Benchmark	Cycles	OF Stalls (Data)	Wrong Branch Entries
`fibonacci`	—	—	—
`descending`	—	—	—
`prime`	—	—	—
`evenorodd`	—	—	—
`palindrome`	—	—	—

Table 5: Template for the required benchmark results table (fill in after running)

12.2 Observations to Comment On

- **Data Hazard Stalls:** Programs with many back-to-back dependent arithmetic instructions (like Fibonacci with `add` followed immediately by using the result) will have more stalls. Load-use hazards (e.g., `load` followed by an instruction using the loaded value) are particularly common.

- **Wrong Branch Entries:** Programs with loops (Fibonacci, descending sort) will have more taken branches since the loop-back branch is taken on every iteration except the last. The descending sort, with its nested loops, will have the most.
- **CPI Analysis:** Ideal CPI for a pipeline is 1.0. The actual CPI will be higher due to stall cycles and flush cycles. You can break down: $\text{CPI} = 1 + \frac{\text{stall cycles+flush cycles}}{\text{total instructions}}$.

13 Common Pitfalls and Debugging Tips

1. **Latch Overwrite Problem:** If you process IF before RW, the data written by IF into the IF_OF latch gets consumed by OF in the same cycle. Always process in reverse order.
2. **Off-by-One in PC:** In the pipelined model, the PC stored with an instruction should be the PC *at the time of fetch*, not the current global PC. Store PC in the latch.
3. **Forgetting to Propagate Bubbles:** If EX has no valid instruction (EX_enable is false), it must set MA_enable to false. Otherwise, the MA stage may process stale data from a previous cycle.
4. **Register x0 Marked Busy:** Never mark x0 as busy. It is hardwired to 0.
5. **Branch + Data Hazard Interaction:** If a branch instruction has a data hazard (its source register is being written by a preceding instruction), the branch must stall. Don't flush on a stalled branch.
6. **Counting Instructions vs. Cycles:** In the pipelined model, instruction count and cycle count are different. Be careful to count only *committed* (completed through RW) instructions, not fetched instructions.
7. **Store Instruction Operand Confusion:** In the ISA, `store` has `rd` as the base address register and `rs1` as the data source. The decoded instruction swaps these in the Operand fields. Be careful when checking for hazards.
8. **End Instruction in Pipeline:** If `end` is fetched but more instructions come after it, those must be discarded or their effects made irrelevant by stopping at RW of `end`.

14 Summary of Changes at a Glance

Complete Change Checklist

1. **Simulator.java**: Reverse stage order, single clock increment per cycle.
2. **RegisterFile.java**: Add `busy[]` array, getter/setter.
3. **IF_OF_LatchType.java**: Add `instructionPC` field.
4. **InstructionFetch.java**: Store PC in latch, keep IF always enabled.
5. **OperandFetch.java**: Add hazard detection, stall logic, busy-bit setting.
6. **Execute.java**: Fix PC calculation (use stored PC), add pipeline flush on taken branch.
7. **MemoryAccess.java**: Propagate bubbles when MA_enable is false.
8. **RegisterWrite.java**: Remove IF re-enable, clear busy bits on writeback.
9. **Statistics.java**: Add `dataHazardStalls`, `wrongBranchCount` counters.
10. **All stages**: Handle enable=false case (propagate bubble to next latch).

A ToyRISC ISA Quick Reference

A.1 Instruction Formats

Format	Bits [31:27]	Bits [26:22]	Bits [21:17]	Bits [16:5]	Bits [4:0]
R3-Type	opcode (5)	rs1 (5)	rs2 (5)	unused (12)	rd (5)
R2I-Type	opcode (5)	rs1 (5)	rd (5)	immediate (17)	
RI-Type	opcode (5)	rd (5)	immediate (22)		

Table 6: ToyRISC instruction format encodings

A.2 Complete Opcode Table

Operation	Opcode	Format	Description
`add`	00000	R3	rd = rs1 + rs2
`addi`	00001	R2I	rd = rs1 + imm
`sub`	00010	R3	rd = rs1 - rs2
`subi`	00011	R2I	rd = rs1 - imm
`mul`	00100	R3	rd = rs1 $\times$ rs2
`muli`	00101	R2I	rd = rs1 $\times$ imm
`div`	00110	R3	rd = rs1 / rs2; x31 = rs1 % rs2
`divi`	00111	R2I	rd = rs1 / imm; x31 = rs1 % imm
`and`	01000	R3	rd = rs1 & rs2
`andi`	01001	R2I	rd = rs1 & imm
`or`	01010	R3	rd = rs1 | rs2
`ori`	01011	R2I	rd = rs1 | imm
`xor`	01100	R3	rd = rs1 $\oplus$ rs2
`xori`	01101	R2I	rd = rs1 $\oplus$ imm
`slt`	01110	R3	rd = (rs1 $<$ rs2) ? 1 : 0
`slti`	01111	R2I	rd = (rs1 $<$ imm) ? 1 : 0
`sll`	10000	R3	rd = rs1 $\ll$ rs2
`slli`	10001	R2I	rd = rs1 $\ll$ imm
`srl`	10010	R3	rd = rs1 $\ggg$ rs2 (logical)
`srli`	10011	R2I	rd = rs1 $\ggg$ imm (logical)
`sra`	10100	R3	rd = rs1 $\gg$ rs2 (arithmetic)
`srai`	10101	R2I	rd = rs1 $\gg$ imm (arithmetic)
`load`	10110	R2I	rd = Mem[rs1 + imm]
`store`	10111	R2I	Mem[rd + imm] = rs1
`jmp`	11000	RI	PC = PC + rd + imm
`beq`	11001	R2I	if rs1 = rd: PC = PC + imm
`bne`	11010	R2I	if rs1 $\neq$ rd: PC = PC + imm
`blt`	11011	R2I	if rs1 $<$ rd: PC = PC + imm
`bgt`	11100	R2I	if rs1 $>$ rd: PC = PC + imm
`end`	11101	RI	Terminate execution

A.3 Registers

Register	Encoding	Purpose
x0	00000	Hardwired zero
x1	00001	Stack pointer
x2	00010	Frame pointer
x3–x30	00011–11110	General purpose
x31	11111	Special (overflow, remainder, shifted-out bits)
PC	—	Program counter

Table 8: ToyRISC register file

B Sample Benchmark: Fibonacci Walkthrough

The Fibonacci benchmark illustrates common hazards:

```
1 loop:
2     add  %x3, %x4, %x5     ; x5 = x3 + x4
3     store %x5, 0, %x7      ; store x5 -> uses x5 (DATA HAZARD!)
4     subi %x7, 1, %x7       ; x7 = x7 - 1
5     addi %x8, 1, %x8       ; x8 = x8 + 1
6     add  %x0, %x4, %x3     ; x3 = x4
7     add  %x0, %x5, %x4     ; x4 = x5 (DATA HAZARD on x5!)
8     jmp  for                ; CONTROL HAZARD
```

Listing 18: fibonacci.asm (excerpt)

Data hazards:

- `store %x5` depends on `add ..., %x5` (1 instruction apart).
- `add ..., %x4` at line 6 depends on `add ..., %x3` which writes x3 (though x4 is the source here, it was written 5 instructions prior, so no hazard).

Control hazard:

- `jmp for` causes a flush every iteration (unconditional jump is always taken).